

PART 1: INSTALLATION

- 1. Install OWASP ZAP.**
- 2. Launch the application.**
- 3. Explore:**
 - **Dashboard**
 - **Sites**
 - **Alerts**
 - **History**
 - **Quick Start**

Introduction

OWASP ZAP (Zed Attack Proxy) is a free and open-source web application security testing tool developed by the OWASP Foundation. It helps security professionals and students identify vulnerabilities in web applications by intercepting and analyzing web traffic.

ZAP can perform:

- Web application scanning
- Spidering (crawling websites)
- Vulnerability detection
- Request and response analysis
- Security testing and penetration testing

Installation Steps

Step 1: Download OWASP ZAP

- Open your browser.
- Visit: [OWASP ZAP Official Website](https://owasp.org/zap/)
- Download the Windows Installer (.exe).

Observation:

The latest stable version of OWASP ZAP was downloaded successfully.

Step 2: Install OWASP ZAP

- Double-click the downloaded installer.
- Click **Next**.
- Accept the License Agreement.
- Choose the installation location.
- Click **Install**.
- Wait for installation to complete.
- Click **Finish**.

Observation:

OWASP ZAP was installed successfully on the system.

Step 3: Launch OWASP ZAP

- Open OWASP ZAP from the Start Menu or Desktop Shortcut.
- A session dialog box appears.
- Select:
Persist Session → Create New Session
- Click **Start**.

Observation:

OWASP ZAP launched successfully and displayed the main interface.

Exploring the OWASP ZAP Interface

1. Dashboard

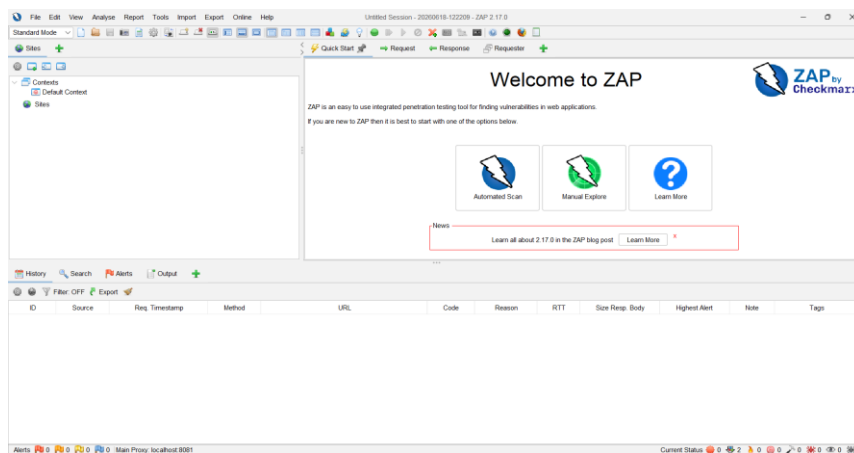
Purpose:

- Displays scan status
- Shows alerts summary
- Provides quick access to common actions

Observation:

The Dashboard provides an overview of security testing activities and vulnerability statistics.

Screenshot:



2. Sites Panel

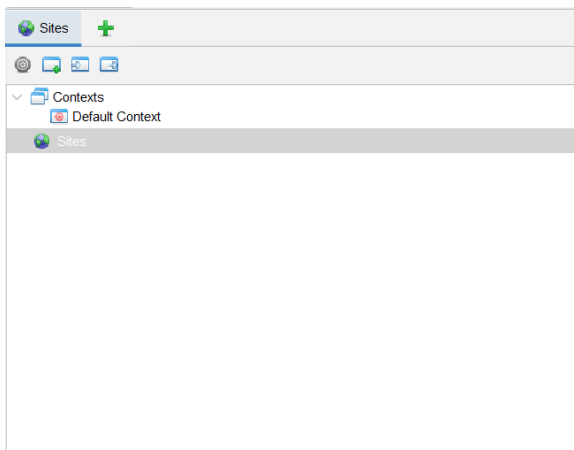
Purpose:

- Displays websites accessed through ZAP
- Shows discovered URLs and page hierarchy
- Helps visualize site structure

Observation:

The Sites panel organizes discovered web pages in a tree structure.

Screenshot :



3. Alerts Panel

Purpose:

- Displays vulnerabilities found during scanning
- Categorizes alerts by risk level

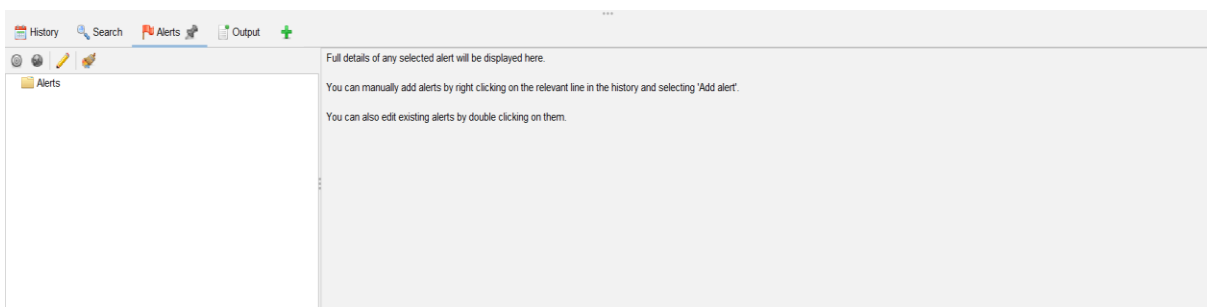
Risk Levels:

- High
- Medium
- Low
- Informational

Observation:

The Alerts panel provides detailed information about detected security issues.

Screenshot :



4. History Panel

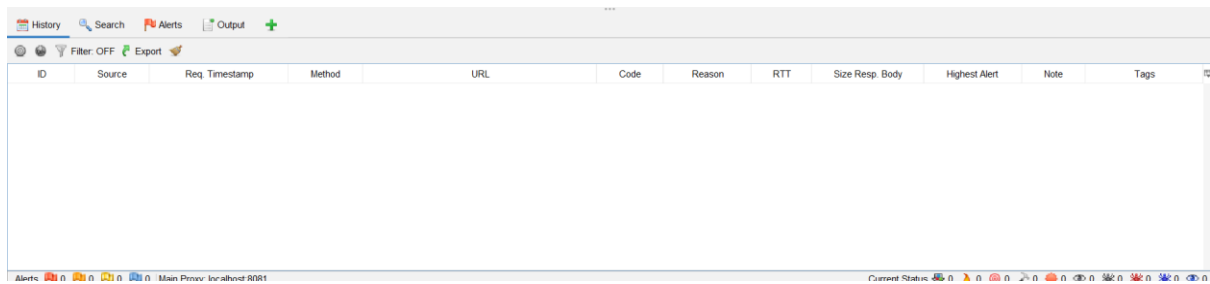
Purpose:

- Stores all intercepted HTTP/HTTPS requests
- Records responses from web servers

Observation:

The History panel captures all communication between the browser and web server.

Screenshot



5. Quick Start

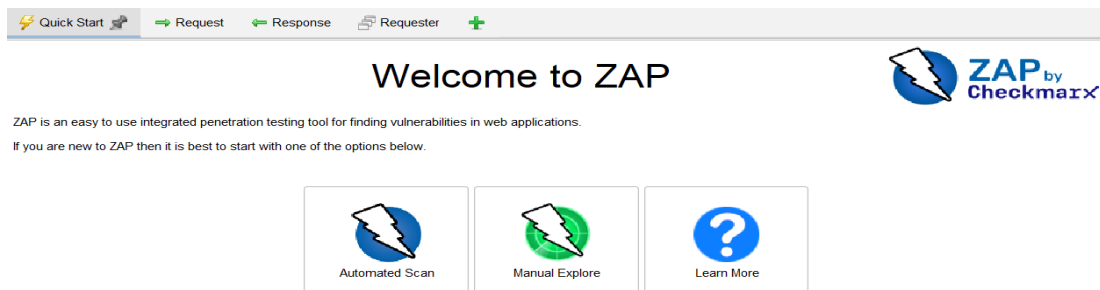
Purpose:

- Open a URL through ZAP
- Launch automated scans
- Start manual testing

Observation:

Quick Start provides easy access to common testing features.

Screenshot



Result

OWASP ZAP was successfully installed and launched. The main components—Dashboard, Sites, Alerts, History, and Quick Start—were explored and their functions understood. The tool is ready for web application security assessment and vulnerability testing.

PART 2: WEBSITE ANALYSIS

Choose a practice website or your own application.

Tasks:

1. Access the website through OWASP ZAP.
2. Explore multiple pages.
3. Observe requests and responses.
4. Create a site tree.

Record:

- Number of pages discovered
- Number of requests captured
- Website structure

Step 1: Open the Target Website Through OWASP ZAP

Procedure

1. Launch **OWASP ZAP**.
2. On the Dashboard, locate the **Quick Start** section.
3. Click **Manual Explore** (or **Open URL**, depending on your ZAP version).
4. In the URL field, enter a target website such as:
`https://demo.owasp-juice.shop/`
5. Click **Launch Browser** or **Open URL**.
6. The website will open in the browser configured by ZAP.
7. Browse the website normally.

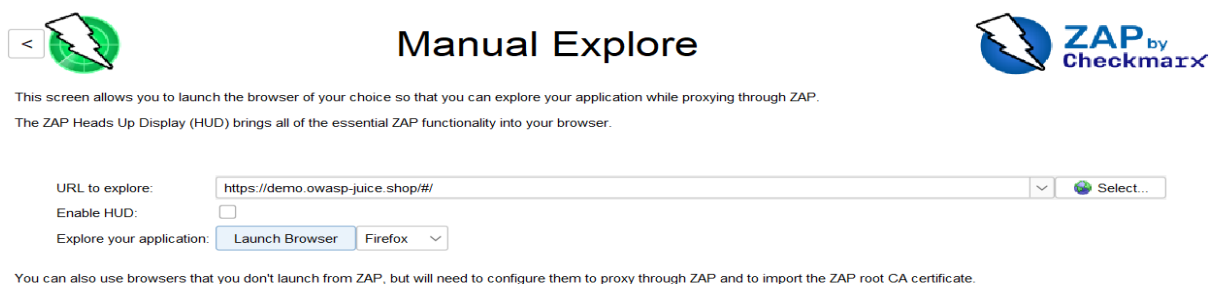
Observation

The website opened successfully through OWASP ZAP. As pages were accessed, HTTP/HTTPS requests began appearing in the **History** tab, and the website started appearing in the **Sites** panel.

Result

The target website was successfully connected through OWASP ZAP, enabling traffic monitoring and website analysis.

Screenshot :



Step 2: Browse Multiple Pages

After opening the website through OWASP ZAP:

1. Browse different sections of the website.
2. Click on various links and menu options.
3. Visit pages such as:
 - Home Page
 - Login Page
 - Registration Page
 - About Us page
 - Registration page
 - AI Chat page
 - Customer Feedback page
 - Score Card page
 - Photo wall page
 - Your Basket page
4. Interact with the website by:
 - Clicking buttons
 - Searching for products
 - Opening product details
 - Navigating between pages

Observation

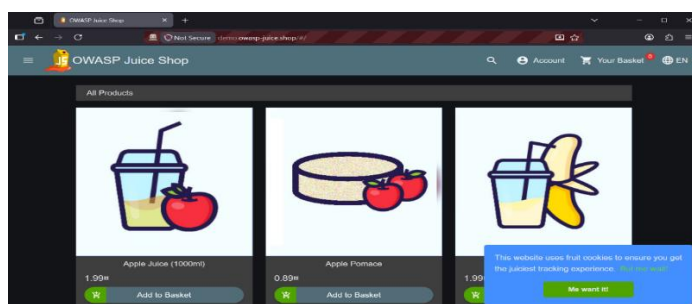
As different pages were visited, OWASP ZAP automatically captured the HTTP/HTTPS requests generated by the browser. New entries appeared in both the **History** tab and the **Sites** tree. The number of discovered pages increased as more areas of the website were explored.

Result

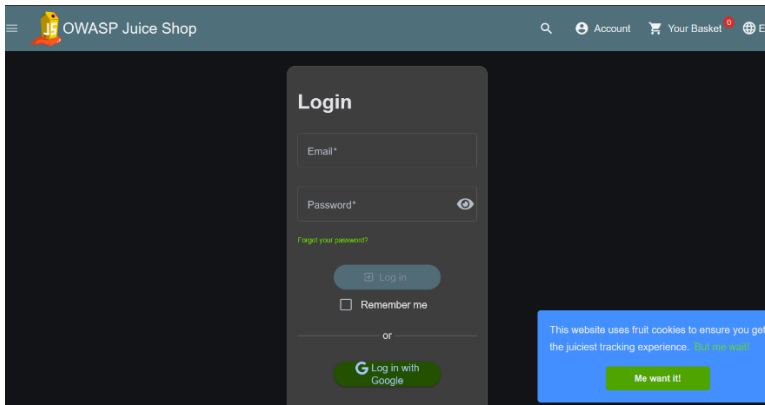
Multiple pages of the target website were successfully explored, allowing OWASP ZAP to collect website structure information and capture network traffic for further analysis.

Screenshot :

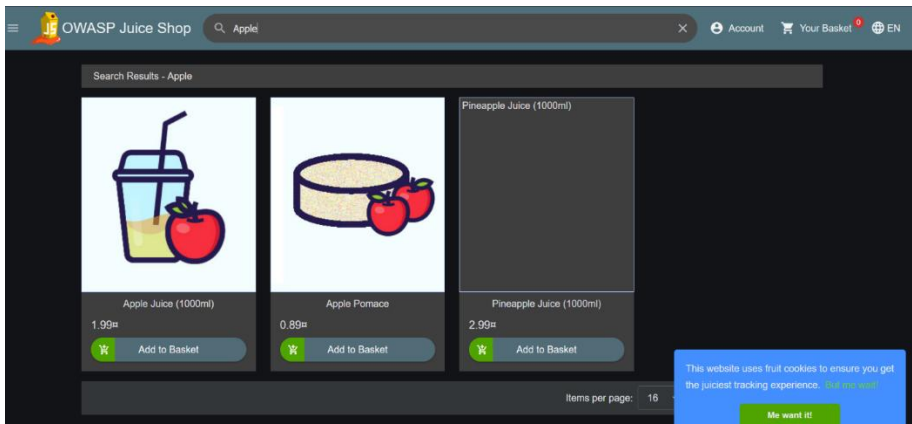
- Home Page



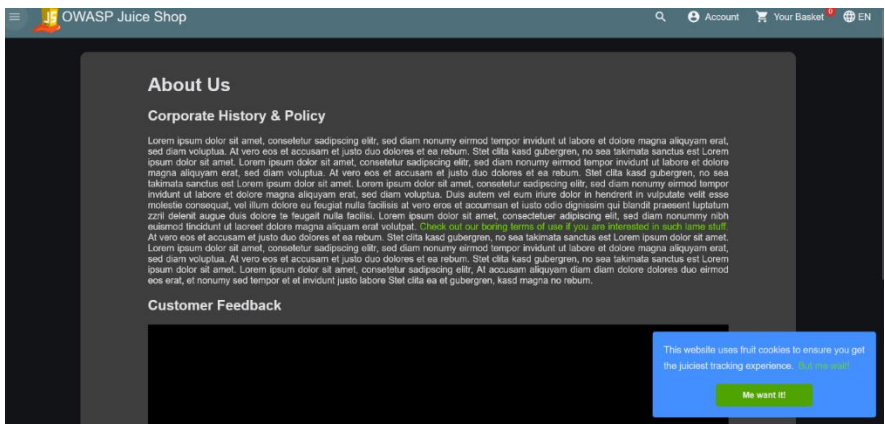
- Login Page



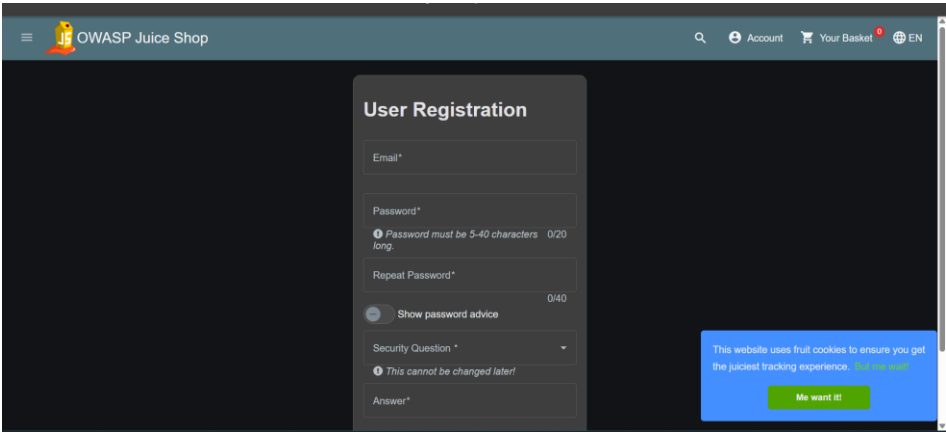
- Products/Search Page



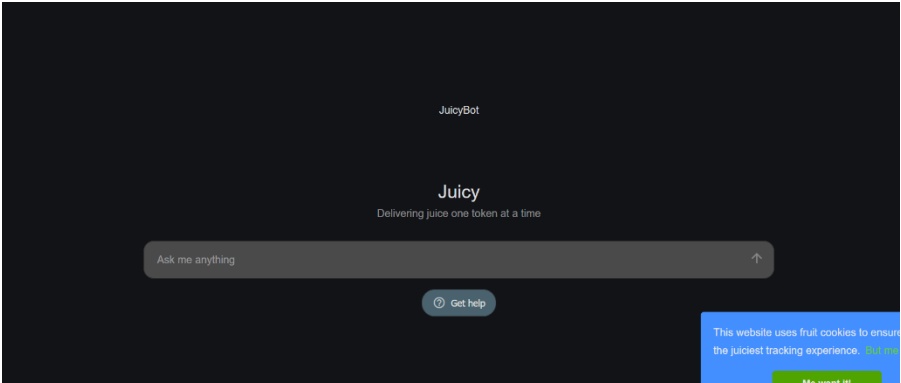
- About Us page



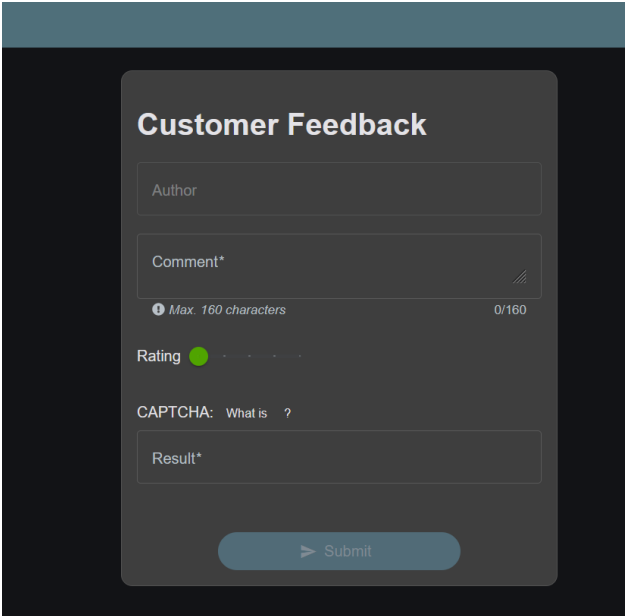
- Registration page



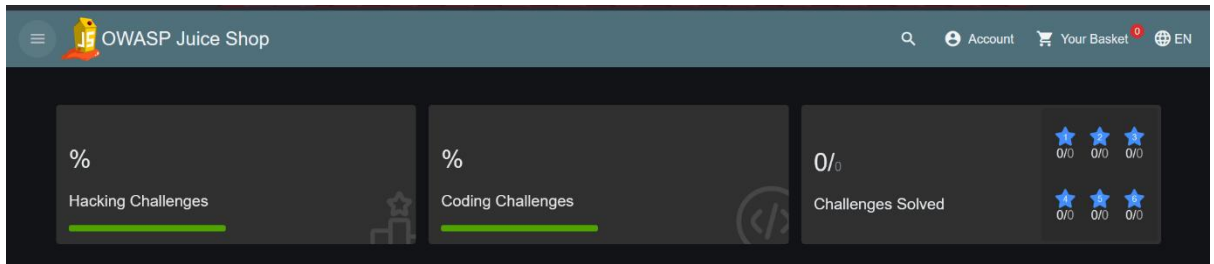
- AI Chat page



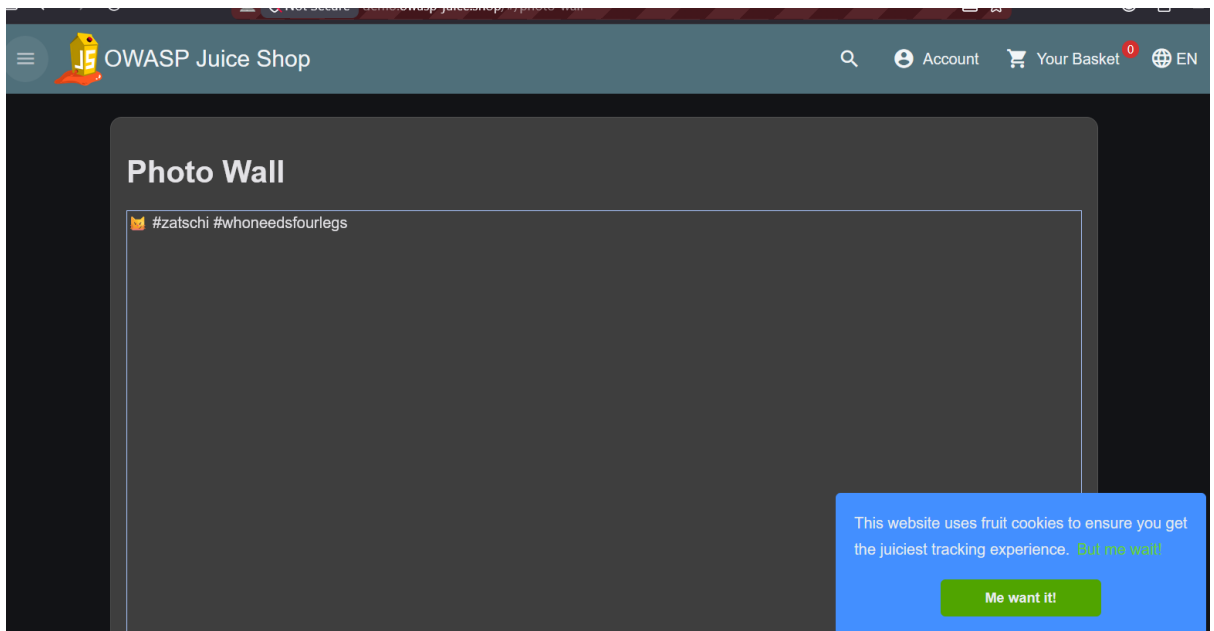
- Customer Feedback page



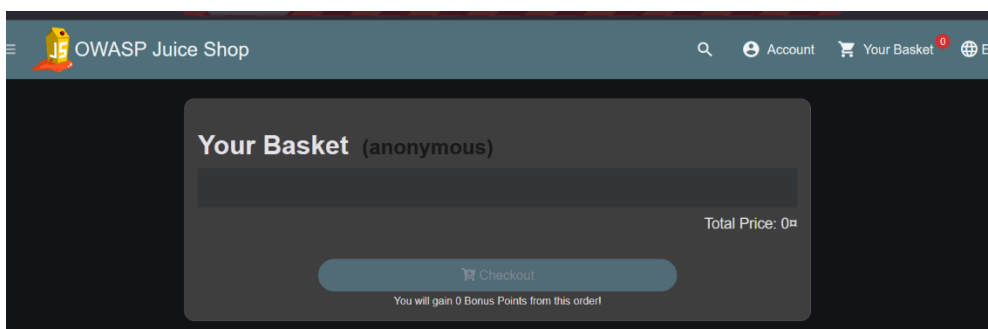
- Score Card page



- Photo wall page



- Your Basket page



Step 3: Observe Requests and Responses

Procedure

1. In OWASP ZAP, click on the **History** tab (bottom-left panel).
2. Browse a few pages on the target website if you have not already done so.
3. In the History tab, select any captured request.

4. The request details will appear in the right panel.
5. Observe:
 - Request Method (GET/POST)
 - URL Requested
 - Request Headers
 - Request Parameters
6. Click the **Response** tab.
7. Observe:
 - Response Status Code (200, 404, etc.)
 - Response Headers
 - HTML/JSON Content Returned

Example Request

GET https://demo.owasp-juice.shop/

Request Method: GET

Purpose: Retrieve the homepage from the server.

Observations

- Every page visit generated one or more HTTP requests.
- Most page loads used the **GET** method.
- Login and registration forms typically use the **POST** method.
- The server responded with status codes indicating the result of each request.
- All captured traffic was recorded in the History panel.

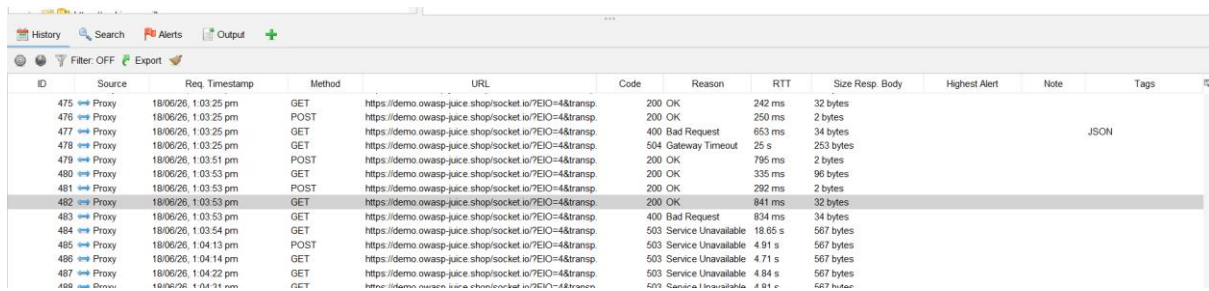
Observation Table

Parameter	Observation
Request Method	GET / POST
Status Code	200 OK
Protocol	HTTPS
Requests Captured	Multiple
Response Received	Successful

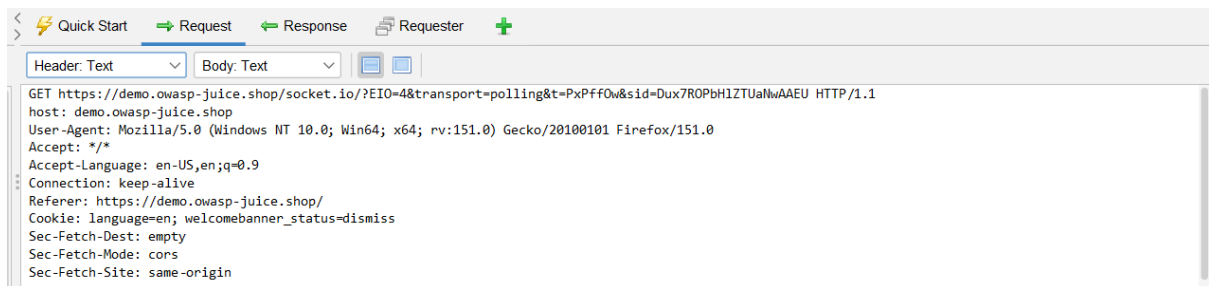
Result

The HTTP requests and responses exchanged between the browser and the web server were successfully observed using OWASP ZAP. The captured traffic provided information about website communication, request methods, URLs accessed, and server responses.

Screenshot :



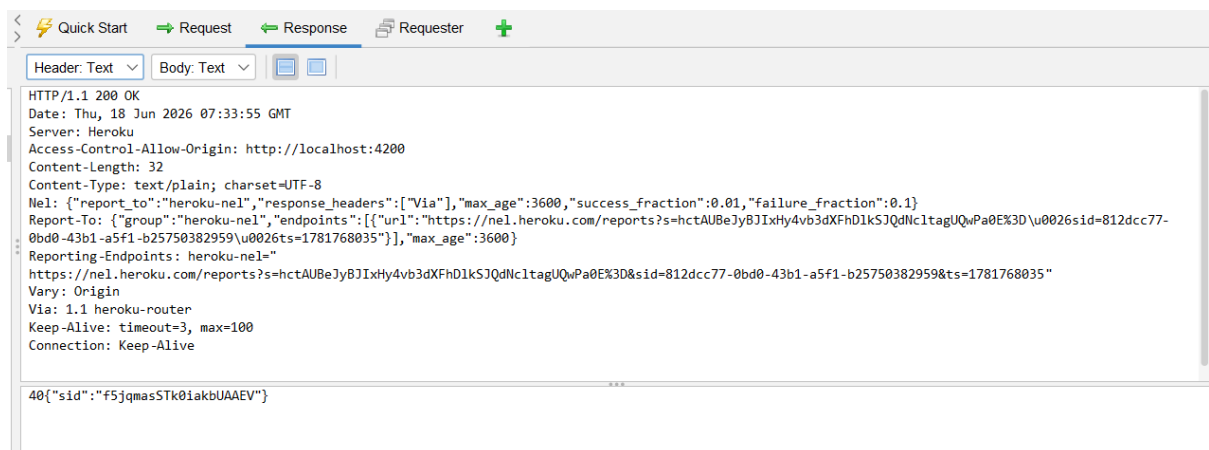
ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note	Tags
475	Proxy	18/06/26, 1:03:25 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	200	OK	242 ms	32 bytes			
476	Proxy	18/06/26, 1:03:25 pm	POST	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	200	OK	250 ms	2 bytes			
477	Proxy	18/06/26, 1:03:25 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	400	Bad Request	653 ms	34 bytes			JSON
478	Proxy	18/06/26, 1:03:25 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	504	Gateway Timeout	25 s	253 bytes			
479	Proxy	18/06/26, 1:03:51 pm	POST	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	200	OK	795 ms	2 bytes			
480	Proxy	18/06/26, 1:03:53 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	200	OK	335 ms	96 bytes			
481	Proxy	18/06/26, 1:03:53 pm	POST	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	200	OK	292 ms	2 bytes			
482	Proxy	18/06/26, 1:03:53 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	200	OK	841 ms	32 bytes			
483	Proxy	18/06/26, 1:03:53 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	400	Bad Request	834 ms	34 bytes			
484	Proxy	18/06/26, 1:03:54 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	503	Service Unavailable	18.65 s	567 bytes			
485	Proxy	18/06/26, 1:04:13 pm	POST	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	503	Service Unavailable	4.91 s	567 bytes			
486	Proxy	18/06/26, 1:04:14 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	503	Service Unavailable	4.71 s	567 bytes			
487	Proxy	18/06/26, 1:04:22 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	503	Service Unavailable	4.84 s	567 bytes			
488	Proxy	18/06/26, 1:04:31 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=	503	Service Unavailable	4.91 s	567 bytes			



Quick Start Request Response Requester +

Header: Text Body: Text

```
GET https://demo.owasp-juice.shop/socket.io/?EIO=4&transport=polling&t=PxFf0w&sid=Dux7R0PbH1ZTUaHAAEU HTTP/1.1
host: demo.owasp-juice.shop
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:151.0) Gecko/20100101 Firefox/151.0
Accept: */*
Accept-Language: en-US,en;q=0.9
Connection: keep-alive
Referer: https://demo.owasp-juice.shop/
Cookie: language=en; welcomebanner_status=dismiss
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
```



Quick Start Request Response Requester +

Header: Text Body: Text

```
HTTP/1.1 200 OK
Date: Thu, 18 Jun 2026 07:33:55 GMT
Server: Heroku
Access-Control-Allow-Origin: http://localhost:4200
Content-Length: 32
Content-Type: text/plain; charset=UTF-8
Nel: {"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Report-To: {"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=hctAUBeJy8JIXHy4vb3dXFhD1k5JQdNc1tagUQwPa0E%3D\u0026sid=812dcc77-0bd0-43b1-a5f1-b25750382959\u0026ts=1781768035"}],"max_age":3600}
Reporting-Endpoints: heroku-nel="https://nel.heroku.com/reports?s=hctAUBeJy8JIXHy4vb3dXFhD1k5JQdNc1tagUQwPa0E%3D&sid=812dcc77-0bd0-43b1-a5f1-b25750382959&ts=1781768035"
Vary: Origin
Via: 1.1 heroku-router
Keep-Alive: timeout=3, max=100
Connection: Keep-Alive

40{"sid":"F5jqmasStk0iakbUAAEV"}
```

Step 4: Create Site Tree

Procedure

1. After browsing multiple pages of the website, return to OWASP ZAP.
2. Locate the **Sites** panel on the left side of the interface.
3. Expand the target website by clicking the + icon next to the domain name.
4. Observe the discovered pages, directories, and resources.
5. Continue expanding subfolders to view the complete website structure.

Observations

- OWASP ZAP automatically created a hierarchical tree structure of the website.
- Each visited page appeared under the target domain.
- The Sites panel helped identify available pages and application modules.

- The website structure became clearer as more pages were explored.

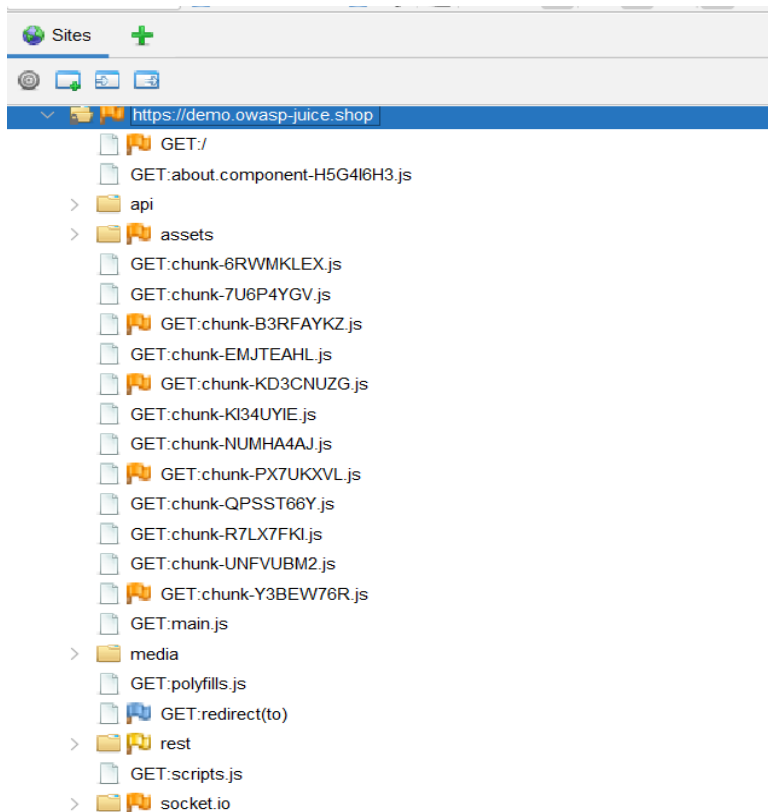
Website Structure Identified

Resource Type	Examples
Main Page	GET: /
API Endpoints	api
Static Assets	assets
Media Files	media
REST Services	rest
WebSocket Resources	socket.io
JavaScript Files	main.js, polyfills.js, chunk-*.js

Result

The website structure was successfully generated and viewed using the Sites panel in OWASP ZAP. The Site Tree provided a clear hierarchical representation of the application's pages and resources, helping in understanding the overall architecture of the target website.

Screenshot :



PART 3: SPIDERING

Tasks:

1. **Run Spider.**
2. **Allow it to discover pages.**
3. **Observe:**
 - **New URLs discovered**
 - **Forms discovered**
 - **Parameters discovered**

Take screenshots.

Step 1: Start Spider Scan

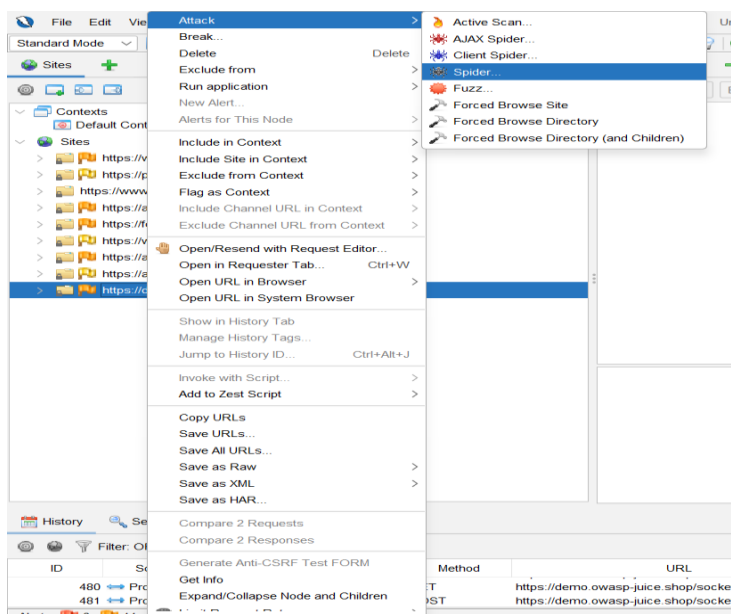
Objective

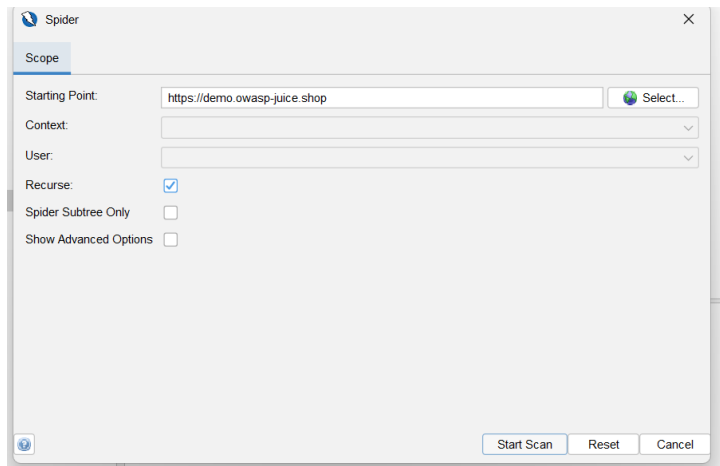
To use OWASP ZAP's Spider feature to automatically crawl the website and discover hidden pages, URLs, forms, and parameters.

Procedure

- Open **OWASP ZAP**.
- In the **Sites** panel, locate the target website:
- <https://demo.owasp-juice.shop>
- Right-click on the website/domain name.
- Select:
Attack → Spider...
- The Spider dialog box will appear.
- Leave the default settings unchanged.
- Click **Start Scan**.

Screenshot :





Step 2: Monitor Spider Progress

Procedure

1. After starting the Spider scan, click the **Spider** tab at the bottom of OWASP ZAP.
2. Observe the scan status and progress percentage.
3. Monitor the following information:
 - a. Number of URLs discovered
 - b. Number of requests sent
 - c. Resources identified
 - d. Scan completion percentage
4. Allow the Spider to continue until it reaches **100% completion**.

Observation

The Spider automatically crawled the website and followed available links. As the scan progressed, new URLs and resources were continuously added to the **Sites** panel. This process helped identify pages that may not have been manually visited during browsing.

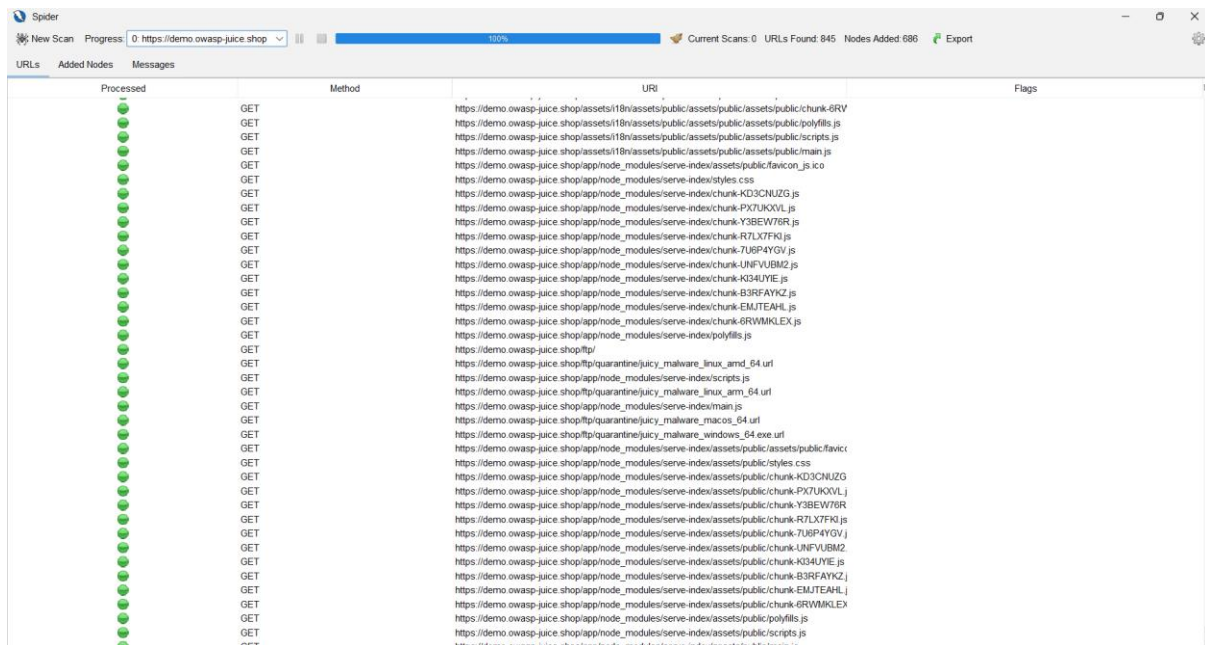
Observation Table

Parameter	Observation
Spider Status	Completed
Progress	100%
URLs Found	845
Nodes Added	686
Current Scans	0 (Completed)
Website Crawled	Successfully

Result

The Spider scan successfully crawled the target website and discovered additional resources. The progress was monitored through the Spider tab, which provided real-time information about the crawling process.

Screenshot :



Step 3: Review Discovered URLs

Procedure

1. After the Spider scan reaches **100% completion**, open the **Spider** tab.
2. Review the list of discovered URLs.
3. Expand the **Sites** panel and compare it with the structure before spidering.
4. Note the newly identified pages, API endpoints, and resources.

Observation

The Spider scan discovered several additional URLs and application resources that were not manually visited. Many of these resources belong to the application's REST API and backend services.

Statistics from Spider Scan

Parameter	Value
URLs Found	845
Nodes Added	686
Spider Status	Completed

Parameter	Value
Progress	100%

Step 4: Identify Forms

Procedure

1. Review the pages discovered by the Spider scan.
2. Browse the OWASP Juice Shop website.
3. Look for pages that accept user input.
4. Identify Forms/Features.
5. Note the parameters associated with these forms.

Forms/Features Identified

Form Name	Purpose
Login Form	User authentication and account access
Registration Form	New user account creation
Search Feature	Product search functionality
Customer Feedback Form	Submit reviews and feedback

Observation

The Spider scan and manual browsing identified multiple forms that accept user input. These forms interact with backend services and process potentially sensitive information such as usernames, passwords, email addresses, and search queries.

Security Relevance

These forms are common targets for attacks such as:

- SQL Injection
- Cross-Site Scripting (XSS)
- Brute Force Attacks
- Credential Stuffing
- Input Validation Bypass

Therefore, they should be tested carefully during a security assessment.

Step 5: Identify Parameters

Procedure

1. Open the **Spider** tab and review the discovered URLs.
2. Open the **History** tab.
3. Select requests containing:
 - a. Query strings (?parameter=value)
 - b. Form submissions
 - c. API calls
4. Observe the parameters present in the Request tab.
5. Record the identified parameters.

Parameters Discovered

Based on the URLs found in OWASP Juice Shop, the following parameters were identified:

Parameter	Purpose
q	Product search query
email	User email address
password	User password
username	User login name
id	Product/User identifier
fields	API response field selection
comment	User feedback
rating	Product review rating

Observation

The Spider scan identified multiple parameters used in search functionality, user management APIs, and review systems. These parameters accept user input and should be validated properly to prevent attacks.

Summary Table

Item	Count
URLs Discovered	845
Nodes Added	24
Forms/Features Discovered	4

Item	Count
Parameters Discovered	8

Screenshot:

ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note	Tags
1,761	Proxy	18/06/26, 1:47:16 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transp=	400	Bad Request	913 ms	34 bytes			JSON
1,762	Proxy	18/06/26, 1:47:16 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transp=	504	Gateway Timeout	24.46 s	253 bytes			
1,763	Proxy	18/06/26, 1:47:40 pm	POST	https://demo.owasp-juice.shop/socket.io/?EIO=4&transp=	200	OK	882 ms	2 bytes			
1,764	Proxy	18/06/26, 1:47:41 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transp=	200	OK	326 ms	96 bytes			
1,765	Proxy	18/06/26, 1:47:42 pm	POST	https://demo.owasp-juice.shop/socket.io/?EIO=4&transp=	200	OK	252 ms	2 bytes			
1,766	Proxy	18/06/26, 1:47:42 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transp=	400	Bad Request	257 ms	34 bytes			JSON
1,767	Proxy	18/06/26, 1:47:42 pm	GET	https://demo.owasp-juice.shop/socket.io/?EIO=4&transp=	200	OK	658 ms	32 bytes			
1,768	Proxy	18/06/26, 1:47:55 pm	POST	https://demo.owasp-juice.shop/api/Feedbacks/	201	Created	836 ms	169 bytes			JSON
1,770	Proxy	18/06/26, 1:47:56 pm	GET	https://demo.owasp-juice.shop/rest/user/whoami?fields=	200	OK	348 ms	11 bytes			
1,771	Proxy	18/06/26, 1:47:56 pm	GET	https://demo.owasp-juice.shop/rest/captcha/	200	OK	922 ms	49 bytes			JSON
1,773	Proxy	18/06/26, 1:47:56 pm	GET	https://demo.owasp-juice.shop/rest/captcha/	504	Gateway Timeout	24.4 s	253 bytes			

Untitled Session - 20260618-134133 - ZAP 2.17.0

Quick Start Request Response Requester

Header: Text Body: Text

POST https://demo.owasp-juice.shop/api/Feedbacks/ HTTP/1.1
host: demo.owasp-juice.shop
Connection: keep-alive
Content-Length: 71
sec-ch-ua-platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36
Accept: application/json, text/plain, */*
sec-ch-ua: "Google Chrome";v="149", "Chromium";v="149", "Not(A)Brand";v="24"
Content-Type: application/json
sec-ch-ua-mobile: ?0
Origin: https://demo.owasp-juice.shop
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://demo.owasp-juice.shop/
Accept-Language: en-US,en;q=0.9
Cookie: language=en; welcomebanner_status=dismiss

{"captchaId":4,"captcha":"31","comment":"fhchv (anonymous)","rating":2}

Untitled Session - 20260618-134133 - ZAP 2.17.0

Quick Start Request Response Requester

Header: Text Body: Text

HTTP/1.1 201 Created
Date: Thu, 18 Jun 2026 08:17:56 GMT
Server: Heroku
Access-Control-Allow-Origin: *
Content-Length: 169
Content-Type: application/json; charset=utf-8
Etag: W/"a9-Bya6L91HpFlq/vQktbAB3A/bnpQ"
Feature-Policy: payment 'self'
Location: /api/Feedbacks/9
Nel: {"report_to":"heroku-nel","response_headers":{"Via"},"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Report-To: {"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=KBPK%2BM06Q16gh4C3MxsPd9vXZUQdcvTL3k1mnc12qr8%3D%u0026sid=812dcc77-0bd0-43b1-a5f1-b25750382959%u0026ts=1781770676"}],"max_age":3600}
Reporting-Endpoints: heroku-nel="https://nel.heroku.com/reports?s=KBPK%2BM06Q16gh4C3MxsPd9vXZUQdcvTL3k1mnc12qr8%3D%u0026sid=812dcc77-0bd0-43b1-a5f1-b25750382959&ts=1781770676"
Vary: Accept-Encoding
Via: 1.1 heroku-router
X-Content-Type-Options: nosniff

{"status":"success","data":{"id":9,"comment":"fhchv (anonymous)","rating":2,"updatedAt":"2026-06-18T08:17:56.748Z","createdAt":"2026-06-18T08:17:56.748Z","userId":null}}

PART 4: ALERT ANALYSIS

Review alerts generated by OWASP ZAP.

For each alert record:

- **Alert Name**
- **Risk Level**
- **Description**
- **Possible Impact**
- **Recommendation**

Objective

To review and analyze the security alerts generated by OWASP ZAP after spidering and scanning the target website.

Step 1: Open the Alerts Panel

Procedure

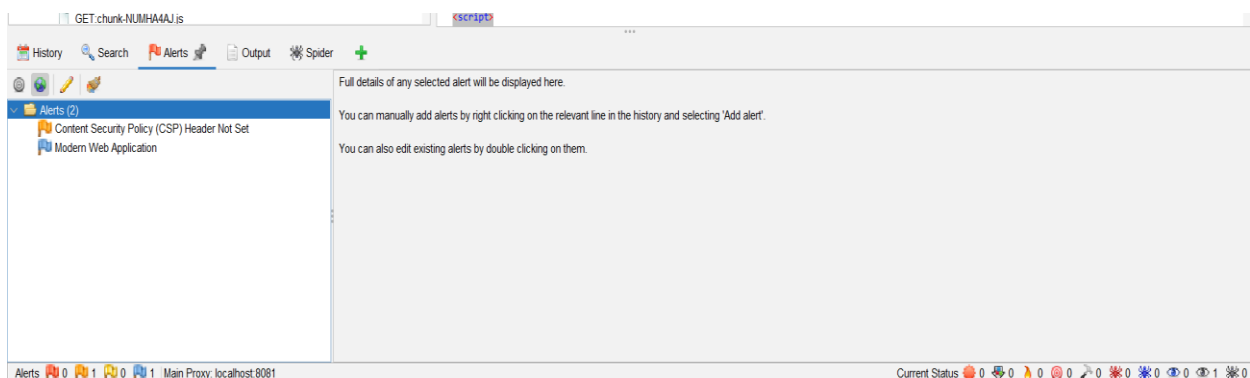
1. In OWASP ZAP, locate the **Alerts** tab (usually on the left panel).
2. Expand the target website.
3. View the generated alerts categorized by risk level:
 - a. High
 - b. Medium
 - c. Low
 - d. Informational

Observation

The Alerts panel displays vulnerabilities and security issues identified during the scan. Each alert contains:

- Alert Name
- Risk Level
- Description
- Solution/Recommendation

Screenshot :



Step 2: Analyze Each Alert

Alert 1: Content Security Policy (CSP) Header Not Set

Risk Level: Medium

Confidence: High

Description:

The application does not implement a **Content Security Policy (CSP)** header. CSP is an additional security mechanism that helps prevent Cross-Site Scripting (XSS) and other code injection attacks by restricting which content sources browsers are allowed to load.

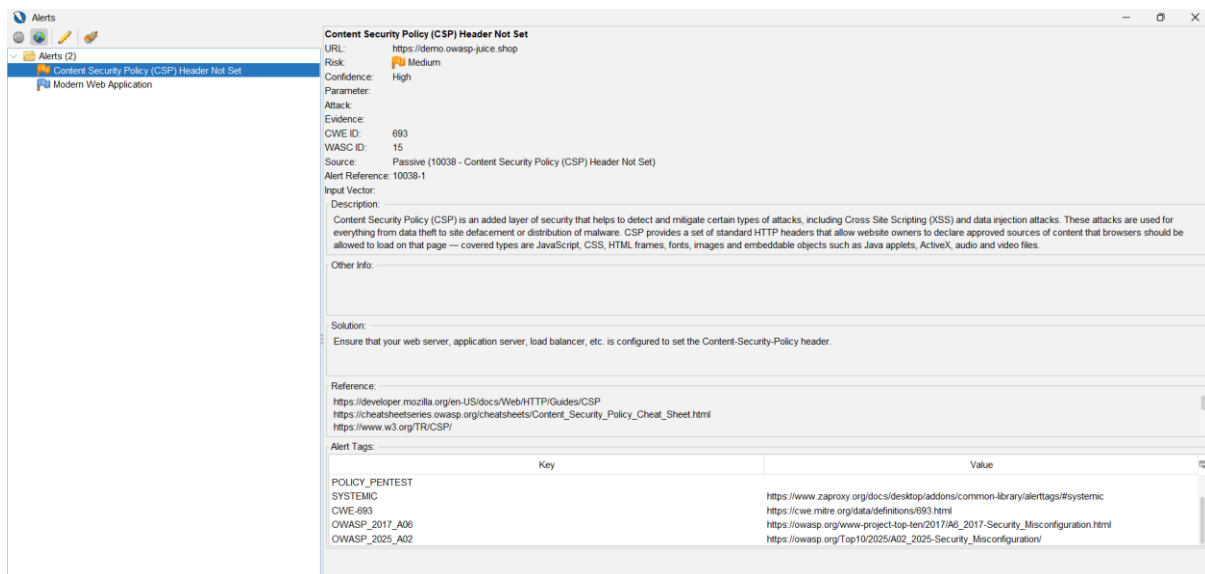
Possible Impact:

- Cross-Site Scripting (XSS)
- Data theft
- Malicious script execution
- Website defacement

Recommendation:

Configure the web server to include an appropriate **Content-Security-Policy** header and restrict trusted sources for scripts, styles, images, and other resources.

Screenshot :



Alert 2: Modern Web Application

Risk Level: Informational

Confidence: Medium

Description:

OWASP ZAP identified the target as a modern JavaScript-based web application. Such

applications often generate content dynamically, which may not be fully discovered by the standard Spider.

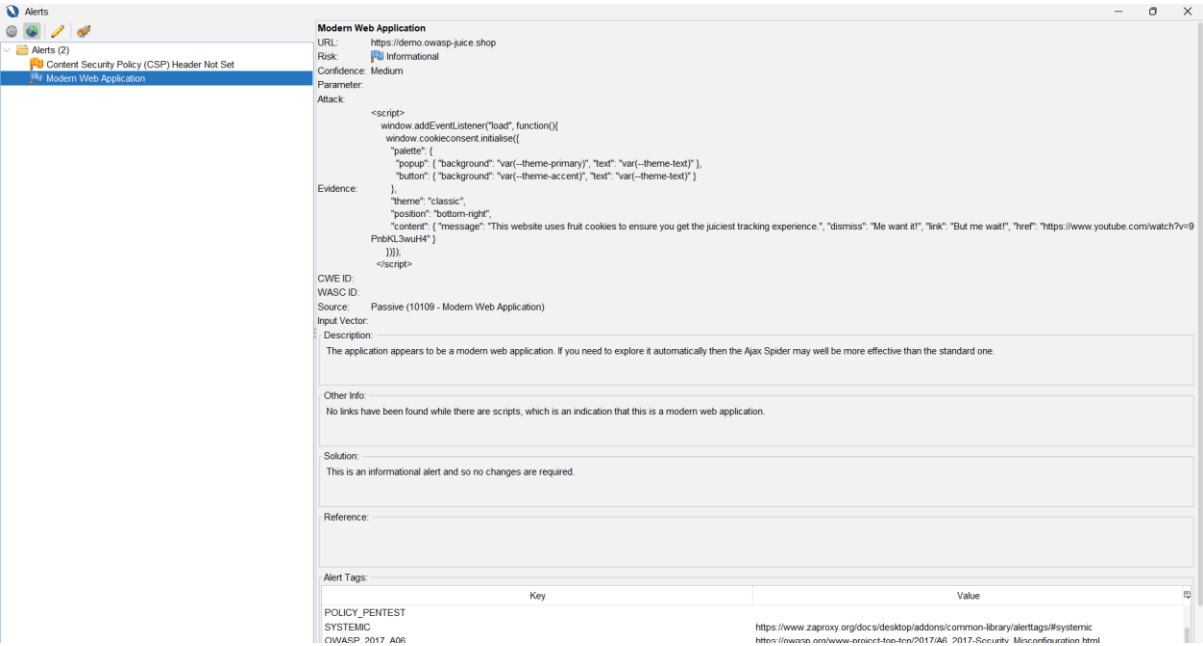
Possible Impact:

- No direct security vulnerability.
- Some pages and resources may remain undiscovered during testing.

Recommendation:

Use the **AJAX Spider** feature in OWASP ZAP to crawl dynamically generated content and improve scan coverage.

Screenshot :



Alert Analysis Table

Alert Name	Risk Level	Description	Possible Impact	Recommendation
Content Security Policy (CSP) Header Not Set	Medium	CSP header missing from responses	Increased risk of XSS and code injection attacks	Configure Content-Security-Policy header
Modern Web Application	Informational	Website uses modern JavaScript technologies	Standard Spider may miss dynamic content	Use AJAX Spider for better coverage

Step 3: Alert Summary

Alert Summary Table

Based on your OWASP ZAP scan results:

Risk Level	Number of Alerts
High	0
Medium	1
Low	0
Informational	1
Total Alerts	2

Detailed Summary

Alert Name	Risk Level
Content Security Policy (CSP) Header Not Set	Medium
Modern Web Application	Informational

Observation

The scan identified a total of **2 alerts**. The primary security concern was the absence of a **Content Security Policy (CSP) Header**, which may increase the risk of Cross-Site Scripting (XSS) attacks. The second alert was informational and indicated that the target website is a modern JavaScript-based application.

PART 5: SECURITY REVIEW

Identify:

- 1. Login Forms
- 2. Search Forms
- 3. Contact Forms
- 4. Registration Forms
- 5. File Upload Features

Create a table containing:

Feature | Security Risk | Recommendation

Step 1: Identify Website Features

Objective

To identify important user-interactive features available in the target web application.

Procedure

- 1. Open the target website (**OWASP Juice Shop**) through OWASP ZAP.
- 2. Browse different pages of the application.
- 3. Observe areas where users can enter data or interact with the application.
- 4. Identify the following features:
 - Login Form
 - Registration Form
 - Search
 - Contact/Feedback Form

Features Identified

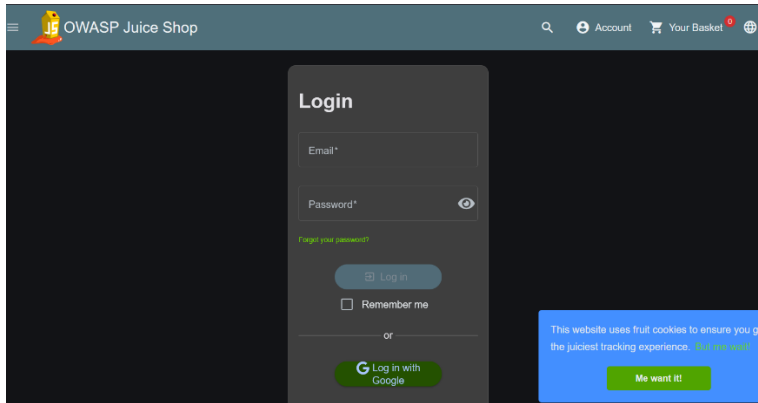
Feature	Status	Purpose
Login Form	Found	User Authentication
Registration Form	Found	New User Registration
Search	Found	Product Search
Feedback Form	Found	User Reviews and Comments
Contact Form	Found	Customer Communication
File Upload Feature	Not Observed	No upload feature identified during testing

Observation

During website exploration, several forms and input fields were identified. These features allow users to authenticate, create accounts, search for products, and submit feedback. Such components process user-supplied data and therefore require proper security controls.

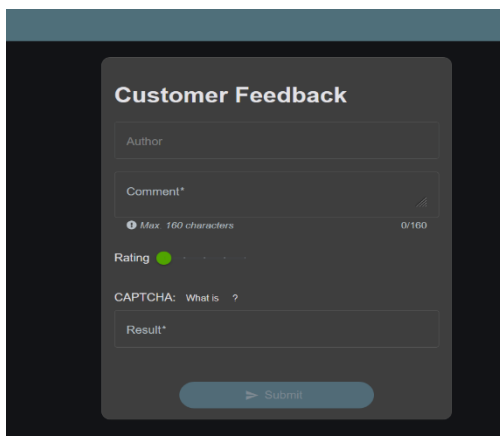
Screenshots :

- Login Form



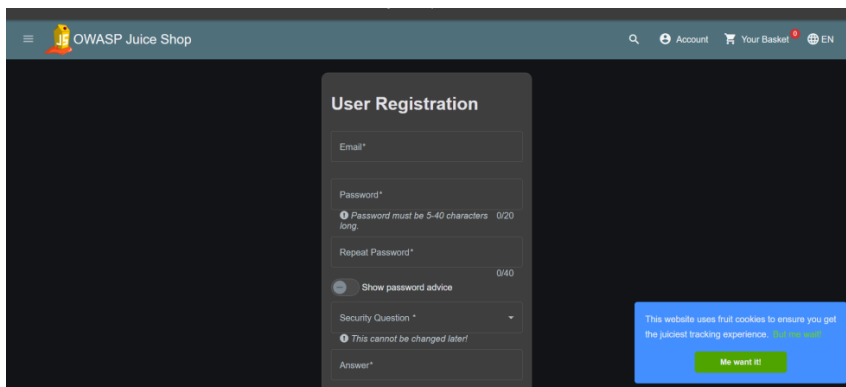
The screenshot shows the 'Login' form on the OWASP Juice Shop website. The form is centered on a dark background. It includes input fields for 'Email*' and 'Password*', a 'Log in' button, a 'Remember me' checkbox, and a 'Log in with Google' button. A link for 'Forgot your password?' is visible. A blue cookie consent banner is at the bottom right.

- Feedback/Contact Form



The screenshot shows the 'Customer Feedback' form. It includes input fields for 'Author', 'Comment*' (with a character count of 0/160), a 'Rating' section with five stars, a CAPTCHA section with the text 'What is ?' and a 'Result*' field, and a 'Submit' button.

- Registration Form



The screenshot shows the 'User Registration' form. It includes input fields for 'Email*', 'Password*' (with a character count of 0/20 and a note 'Password must be 5-40 characters long'), 'Repeat Password*' (with a character count of 0/40), a 'Show password advice' link, a 'Security Question*' dropdown menu (with a note 'This cannot be changed later!'), and an 'Answer*' field. A blue cookie consent banner is at the bottom right.

Step 2: Security Risk Assessment

Security Review Table

Feature	Security Risk	Recommendation
Login Form	Brute-force attacks, credential stuffing, weak passwords	Implement account lockout, CAPTCHA, and Multi-Factor Authentication (MFA)
Registration Form	Weak passwords, fake accounts, automated registrations	Enforce strong password policies and email verification
Search Feature	SQL Injection (SQLi), Cross-Site Scripting (XSS)	Validate and sanitize all user input before processing
Feedback/Contact Form	Spam, malicious input, Cross-Site Scripting (XSS)	Use CAPTCHA, input validation, and output encoding
Session Management	Session hijacking and cookie theft	Enable Secure and HttpOnly cookie flags
REST API Endpoints	Unauthorized access, data exposure	Implement authentication, authorization, and access control

Observation

The identified features process user-supplied data and therefore may be vulnerable to common web attacks if proper security controls are not implemented. Authentication forms, search functionality, and feedback forms are particularly important areas for security testing.

Analysis

- **Login and Registration Forms** handle sensitive user credentials and require strong authentication controls.
- **Search** should validate user input to prevent injection attacks.
- **Feedback/Contact Forms** should sanitize user input to prevent malicious code execution.
- **Session Management** should protect cookies from theft and unauthorized access.
- **API Endpoints** should enforce authentication and authorization checks.

Step 3: Identify Sensitive Information Processed

Procedure

1. Browse the application and review the identified forms.
2. Observe the fields present in:
 - a. Login Form
 - b. Registration Form
 - c. Search Feature
 - d. Feedback Form
3. Analyze the requests captured in the **History** tab.
4. Identify the data being submitted to the server.

Sensitive Information Identified

Information Type	Example
User Credentials	Email Address, Password
Personal Information	Username, Email
Session Information	Authentication Cookies, Session Tokens
Search Queries	Product Search Keywords
User Feedback	Reviews, Comments, Ratings
Account Information	Registration Details

Observation

The application processes several types of user data. The most sensitive information includes login credentials, session cookies, and personal details entered during registration. This information should be protected using encryption and secure session management.

Step 4: Identify Security Controls Present in the Application

Procedure

1. Review the website during browsing and testing.
2. Analyze requests and responses captured in OWASP ZAP.
3. Observe security-related features such as HTTPS, authentication, session management, and security headers.
4. Record the identified security controls.

Security Controls Identified

Security Control	Purpose
HTTPS (TLS Encryption)	Encrypts communication between client and server to protect sensitive data during transmission.
Authentication System	Verifies user identity before granting access to protected resources.
Session Management	Maintains authenticated user sessions using cookies or tokens.
Input Validation	Helps prevent malicious input and injection attacks.
Access Control	Restricts unauthorized users from accessing protected resources.
REST API Structure	Provides controlled communication between frontend and backend services.
CAPTCHA (where applicable)	Prevents automated bot activities and spam submissions.

Observation

The OWASP Juice Shop application uses HTTPS to secure communication and provides authentication mechanisms for user accounts. Session management is implemented through cookies, and user input is processed through various forms and API endpoints. However, the OWASP ZAP scan identified that the **Content Security Policy (CSP) Header** is not configured, indicating an area where security can be improved.

Security Improvement Recommendations

Issue	Recommendation
Missing CSP Header	Implement a Content-Security-Policy header to mitigate XSS attacks.
Session Security	Ensure cookies use Secure and HttpOnly flags.
Input Validation	Continue validating and sanitizing all user input.
Authentication	Consider Multi-Factor Authentication (MFA) for additional protection.

1. How many pages were discovered?

During the Spider scan, **845 pages were discovered** by OWASP ZAP. The scan successfully identified multiple application resources, API endpoints, JavaScript files, and directories within the OWASP Juice Shop website.

2. How many forms exist?

A total of **4 forms/features** were identified:

1. Login Form
2. Registration Form
3. Search Feature
4. Customer Feedback Form

These forms allow users to interact with the application and submit information.

3. What sensitive information might be processed?

The application may process the following sensitive information:

- Usernames
- Email Addresses
- Passwords
- Session Cookies
- Search Queries
- Customer Feedback and Comments
- Account Registration Details

This information should be protected using secure authentication, encryption, and proper access controls.

4. What security controls can you identify?

The following security controls were identified:

- **HTTPS Encryption** – Protects data transmitted between client and server.
- **Authentication System** – Verifies user identity through login functionality.
- **Session Management** – Maintains authenticated user sessions using cookies.
- **Input Validation** – Helps prevent malicious input and injection attacks.
- **Access Control** – Restricts unauthorized access to resources.
- **REST API Security** – Controls communication between frontend and backend services.
- **CAPTCHA** (where applicable) – Helps prevent automated abuse and spam.

These controls contribute to the overall security of the web application.